P0401R1
Providing size feedback in the Allocator interface

Published Proposal, 2019-06-11

This version:
http://wg21.link/P0401R1

Authors:
Jonathan Wakely
Chris Kennelly (Google)

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Utilize size feedback from Allocator to reduce spurious reallocations

Table of Contents

§ 1. Introduction

This is a library paper to describe how size feedback can be used with allocators:

- In the case of `std::allocator`, the language feature proposed in [P0901R3] could be used.
- In the case of custom allocators, the availability of the traits interface allows standard library containers to leverage this functionality.

§ 2. Motivation

Consider code adding elements to `vector`:

```
std::vector<int> v = {1, 2, 3};
// Expected: v.capacity() == 3

// Add an additional element, triggering a reallocation.
v.push_back(4);
```

Many allocators only allocate in fixed-sized chunks of memory, rounding up requests. Our underlying heap allocator received a request for 12 bytes (`3 * sizeof(int)`) while constructing `v`. For several implementations, this request is turned into a 16 byte region.

§ 2.1. Why not `realloc`

`realloc` poses several problems problems:

- We have unclear strategies for growing a container. Consider the two options (`a` and `b`) outlined in the comments of the example below. Assume, as with the §2 Motivation example, that there is a 16 byte size class for our underlying allocator.

  ```
  std::vector<int> v = {1, 2, 3};
  // Expected: v.capacity() == 3

  // Add an additional element.  Does the vector:
  // a) realloc(sizeof(int) * 4), adding a single element
  // b) realloc(sizeof(int) * 6), attempt to double, needing reallocation
  v.push_back(4);

  // Add another element.
  // a) realloc(sizeof(int) * 5), fails in-place, requires reallocation
  // b) size + 1 <= capacity, no reallocation required
  v.push_back(5);
  ```

  Option `a` requires every append (after the initial allocation) try to `realloc`, requiring an interaction with the allocator each time.

  Option `b` requires guesswork to probe the allocator's true size. By doubling (the typical growth pattern for `std::vector` in libc++ and libstdc++), we skip past the existing allocation's size boundary. While we can use a reduced growth rate, doing so requires more interactions with the allocator (calls to `realloc`), as well as copies (when we cannot resize in-place).

  Both of these options add extra branching on the control flow for growing the buffer.

- When the allocator cannot resize the buffer in-place, it allocates new memory and `memcpy`'s the contents. For non-trivially copyable types, this is disastrous. The *language* lacks support for reallocating in-place *without* moving. While this function could be implemented as magic library function, it could not interact with many underlying `malloc` implementations (libc, in the case of libc++ and libstdc++) used to build `operator new`, nor could it interact with user-provided replacement functions.

  In-place reallocation has been considered in [N3495] (throwing `std::bad_alloc` when unable) and [P0894R1] (returning `false` when unable).

For fixed-size allocators it makes more sense, and is much simpler for containers to adapt to, if the allocator is able to over-allocate on the initial request and inform the caller how much memory was made available.

§ 3. Proposal

Wording relative to [N4800].

We propose an optional extension to allocator to return the allocation size.

Table 34 - Cpp17Allocator Requirements

`a.allocate(n)`	`X::pointer`	Memory is allocated for `n` objects of type `T` but objects are not constructed. `allocate` may throw an appropriate exception. [Note: If `n == 0`, the return value is unspecified. — end note]	
`a.allocate_with_size(n)`	`std::pair<X::pointer, X::size_type>`	Memory is allocated for at least `n` objects of type `T` but objects are not constructed and returned as the first value. The actual number of objects `m`, such that `m>=n`, that memory has been allocated for is returned in the second value. `allocate` may throw an appropriate exception. [Note: If `n == 0`, the return value is unspecified. — end note]	`{a.allocate(n), n}`
`a.allocate_with_size(n, y)`	`std::pair<X::pointer, X::size_type>`	Memory is allocated for at least `n` objects of type `T` but objects are not constructed and returned as the first value. The actual number of objects `m`, such that `m>=n`, that memory has been allocated for is returned in the second value. `allocate` may throw an appropriate exception. The use of `y` is unspecified, but is intended as a hint to aid locality. [Note: If `n == 0`, the return value is unspecified. — end note]	`{a.allocate(n), n}`
`a.deallocate(p,n)`	(not used)	*Requires*: `p` shall be a value returned by an earlier call to `allocate` or `allocate_with_size` that has not been invalidated by an intervening call to	

	`deallocate`. If this memory was obtained by a call to `allocate`, `n` shall match the value passed to `allocate` to obtain this memory. Otherwise, `n` shall match the value returned by `allocate_with_size` to obtain this memory. *Throws*: Nothing.

Amend [allocator.traits]:

```
namespace std {
  template struct allocator_traits {
    ...
    [[nodiscard]] static pointer allocate(Alloc& a, size_type n);
    [[nodiscard]] static pointer allocate(Alloc& a, size_type n, const_void_pointer hint
);

    [[nodiscard]] static std::pair allocate_with_size(Alloc& a, size_type n);
    [[nodiscard]] static std::pair allocate_with_size(Alloc& a, size_type n, const_void_
pointer hint);

    ...
  };
}
```

Amend [allocator.traits.members]:

```
[[nodiscard]] static pointer allocate(Alloc& a, size_type n);
```

- *Returns*: `a.allocate(n)`

```
[[nodiscard]] static pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
```

- *Returns*: `a.allocate(n, hint)` if that expression is well-formed; otherwise, `a.allocate(n)`.

```
[[nodiscard]] static std::pair allocate_with_size(Alloc& a, size_type n);
```

- *Returns*: `a.allocate_with_size(n)` if that expression is well-formed, otherwise, `{a.allocate(n), n}`

```
[[nodiscard]] static pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
```

- *Returns*: `a.allocate_with_size(n, hint)` if that expression is well-formed; otherwise, `a.allocate_with_size(n)` if that expression is well-formed: otherwise, `{a.allocate(n), n}`.

§ 4. Design Considerations

§ 4.1. `allocate` selection

There are multiple approaches here:

- Return a pointer-size pair, as presented.
- Overload `allocate` and return via a reference parameter. This potentially hampers optimization depending on ABI, as the value is returned via memory rather than a register.

§ 4.2. `deallocate` changes

We now require flexibility on the size we pass to `deallocate`. For container types using this allocator interface, they are faced with the choice of storing *both* the original size request as well as the provided size (requiring additional auxillary storage), or being unable to reproduce one during the call to `deallocate`.

As the true size is expected to be useful for the `capacity` of a `string` or `vector` instance, the returned size is available without additional storage requirements. The original (requested) size is unavailable, so we relax the `deallocate` size parameter requirements for these allocations.

§ 4.3. Interaction with `polymorphic_allocator`

`std::pmr::memory_resource` is implemented using virtual functions. Adding new methods, such as the proposed allocate API would require taking an ABI break.

§ References

§ Informative References

[N3495]
Ariane van der Steldt. inplace realloc. 7 December 2012. URL: https://wg21.link/n3495

[N4800]
Working Draft, Standard for Programming Language C++. 2019-01-21. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4800.pdf

[P0894R1]
realloc for C++. 2019-01-18. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0894r1.md

[P0901R3]
Size feedback in operator new. 2019-01-21. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0901r3.html